

## 2.36. Le grand rond : 32 × 32

Le même rond en 32 × 32 : Perso ROND\_32, posé avec `sprite32()`. Quatre fois plus de pixels, et une courbe bien plus douce.

### 1. Ce qu'il faut comprendre

**Le grand rond fait 32 × 32 pixels** : 32 rangées de 32 signes, dans `perso_ROND_32.cpp`. Il s'écrit `Perso ROND_32 = {...}`, comme un personnage de 16 × 16 : c'est le **nombre de rangées** qui dit au compilateur sa taille.

**Il n'est pas grossi : il est redessiné.** Le contour a toujours un seul pixel d'épaisseur, et la courbe a deux fois plus de marches, deux fois plus petites. C'est pour ça qu'il paraît bien plus rond que celui du 2.01.

**Il se pose avec `sprite32()`** : `sprite32(0, 64, 40, ROND_32)` prend **16 lutins** (les numéros 0 à 15), en carré de 4 × 4. 64 = (160 – 32) / 2, le milieu. Il faut `#include <sprite32>`.

**Un grand dessin coûte cher** : 16 lutins sur les 40 de la console, et 16 tuiles. Un écran peut en montrer deux, pas dix.

**Dans la galerie**, le bouton **32 × 32** donne toutes les formes à cette taille : ROND\_32, CARRE\_32...

**Essaie** : fais-le descendre, avec une variable `y` qui grandit d'un pixel par image, comme au 35.4.

#### Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <sprite32>` → [35.4. La fonction `sprite32\(\)` — un grand personnage de 32 × 32](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1024 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          57 o ← la console, à l'allumage – la mise en route de la console
main()             405 o ← démarrage – ta fonction
VBlank              8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage       28 o ← image() – attend l'image suivante
AttendreVBlank      16 o ← démarrage – attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte         10 o ← texte() – écrit un texte, lettre par lettre
RangerLutins        11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles        76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage – vide les deux cartes de l'écran
Tuiles              352 o ← CopierTuiles – 6 lettres sur 43, et 16 tuiles dessinées
DonneesTransfert    10 o ← InstallerTransfert – la petite routine qui copie les lutins
"GRAND ROND"        10 o ← texte() – un texte gravé

Lettres gravées : ADGNOR
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
```

Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes  
Fonctions du langage appelées :

```
sprite()   x 16   384 o sur place, dans main()
texte()    x  1    11 o sur place, dans main() + EcrireTexte (10 o)
image()    x  1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() x  4     0 o sur place, dans main()
sprite32() x  1     0 o sur place, dans main()
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- 2.36 Le grand rond : 32 x 32 ----
// Le rond en 32 x 32, au milieu de l'écran.

#include <texte>           // texte() : écrire un mot
#include <sprite32>        // sprite32() : poser un dessin de 32 x 32
#include <Perso>           // Perso : un dessin de 16 x 16 ou de 32 x 32
#include "perso_ROND_32.cpp" // le grand rond, dans son fichier
```

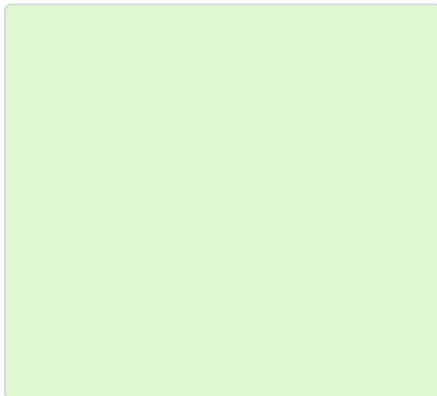
## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite32(0, 64, 40, ROND_32); // les lutins 0 à 15, coin haut-gauche en (64, 40)
    texte(5, 11, "GRAND ROND");   // le nom, sous la forme (qui s'arrête à 72)

    while (true) {
        image();
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

Un grand rond de 32 x 32 au milieu de l'écran, et GRAND ROND dessous.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le grand rond est posé en (64, 40) (64, 40)
- ✓ ses 16 lutins font un carré de 32 (lutin 15 en 88, 64)
- ✓ GRAND ROND est écrit dessous

---

Ensuite : **2.37. Les trois tailles côte à côte** — série 2 : Les formes géométriques. · [gameboy3](#) — écrire une cartouche Game Boy en C++